

SMART CAMPUS PWr

Technical Project Plan

Mobile Applications Development | Wrocław University of Science and
Technology | Summer 2026

Final Submission Deadline: 24 June 2026

1. PROJECT OVERVIEW & CONCEPT

Smart Campus PWr is a comprehensive Android mobile application for Wroclaw University of Science and Technology (PWr), combining the mandatory MAD project requirements with advanced extensions tailored to the academic environment. The application supports three user roles: Administrator, Lecturer, and Student.

The project is developed by a 3-person team across 7 milestone sprints during the 2026 summer semester. This document covers the full technology stack, team responsibilities, Firestore schema, milestone schedule, and a complete feature list — both required and optional.

1.1 Core PWr-Specific Extensions

- PWr AI Chatbot: intelligent assistant with a full university knowledge base (regulations, schedules, FAQs, room locations)
- Interactive campus map: Google Maps with pins for all buildings A1-D3, room search, pedestrian navigation
- Review & rating system: star ratings for courses, lecturers, buildings, and rooms with aggregated averages
- Advanced timetable: weekly view, ICS calendar export, alerts 15 minutes before class
- Course materials hub: lecturer uploads PDFs, videos, links, and notes per course with versioning

1.2 Features (Section 12)

- Contextual GPS alarms: push notification 'leave now' calculated from live location and walking time to room
- Study Match: algorithm pairing students into study groups based on shared courses, grades, schedule, and location
- PWr Exchange: marketplace for notes, old exams, and tutoring ads from senior students
- Live Feedback Dashboard: 30-second post-lecture survey with real-time comprehension charts for the lecturer
- Similarity Check: AI-powered plagiarism detection comparing submissions via text embeddings
- XP + Academic Badges: gamification with points, leaderboards, and badges like 'Calculus Guru'

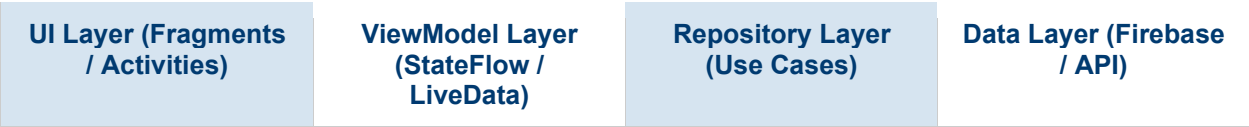
2. TECHNOLOGY STACK

Layer	Technology	Notes
Language / Platform	Kotlin (Android)	Min SDK 26, Target SDK 34+
Architecture	MVVM + Repository Pattern	ViewModel, LiveData/StateFlow, Use Cases
Dependency Injection	Hilt (Dagger)	Injection for ViewModel, Repository, Services
Authentication	Firebase Authentication	Email/password + role-based custom claims

Database	Firebase Firestore	NoSQL, real-time listeners, offline persistence
File Storage	Firebase Storage (Spark)	Course materials, assignment submissions
Push Notifications	Firebase Cloud Messaging	Deadlines, announcements, chat messages
Real-time Chat	Firestore Listeners	messages collection, snapshot listeners in ViewModel
AI Chatbot	Google Gemini 1.5 Flash API	RAG with PWr knowledge base stored in Firestore
Maps	Google Maps SDK for Android	Building pins, indoor navigation, campus heatmap
HTTP / REST	Retrofit + OkHttp	Gemini API, optional backend, weather API
Image Loading	Glide / Coil	Avatars, course thumbnails, material previews
Navigation	Jetpack Navigation Component	Single Activity, role-specific NavGraphs
UI Framework	Material Design 3	PWr colors (navy #003A70 + red #CC0000), dark mode
Serialization	Gson / Kotlinx Serialization	Firestore models, API response parsing
CI/CD	GitHub Actions	Lint, unit tests, automated APK build on push

2.1 Application Architecture – MVVM + Repository

Each layer is independent and testable. ViewModels contain no business logic — they delegate to Use Cases. The Repository is the single source of truth for ViewModels.



3. TEAM RESPONSIBILITIES

Backend & Firebase Architecture

- Firebase project setup: Auth, Firestore, Storage, FCM, Security Rules
- Implementation of Repository layer and Use Cases
- Kotlin data models (data classes, sealed classes for UI states)
- Firestore and Storage security rules
- CI/CD with GitHub Actions, dev/prod environment configuration
- Unit tests for Repository and ViewModels
- XP transaction logic, Similarity Check embedding storage

UI/UX & Course Management

- All screens in Material Design 3 with PWr colors (navy #003A70 + red #CC0000)
- Jetpack Navigation Component (role-specific NavGraphs), bottom navigation
- Course screens: list, details, enrollment, materials, assignments
- Weekly timetable view (custom RecyclerView or CalendarView)
- Firestore to ViewModel to Fragment data binding, pagination, error states
- File uploads (FileProvider, Storage upload progress UI)
- PWr Exchange UI, Live Feedback charts (MPAndroidChart)

AI Chatbot, Maps, Chat & Features

- AI Chatbot: Gemini API integration, RAG pipeline with knowledge Firestore collection
- Campus map: Google Maps SDK, building pins, InfoWindows with room details
- Real-time chat: Firestore listeners, ChatRepository, MessageAdapter
- FCM Service: push notification handling, deep links to chat/assignment
- Review system: ReviewRepository, star rating UI, stats aggregation
- ICS export, QR scanner, GPS alarm logic, Study Match algorithm
- AI Tutor: WorkManager + Gemini daily plan generation

Area	Backend & Auth	UI & Courses	AI, Maps & Chat
Auth & Roles	Firestore Auth, custom claims, session management	Login/register screens, splash, role navigation	—
Firestore	Data models, security rules, indexes	UI-to-Repository binding, pagination	Chat subcollections, knowledge base collection
Courses	CourseRepository, EnrollmentService	CourseListFragment, CourseDetailFragment	—
Assignments	AssignmentRepository, Storage upload/download	AssignmentUI, SubmissionUI, FilePickerIntegration	—
Timetable	ScheduleRepository, schedule data models	Weekly timetable view (TimetableFragment)	ICS export, alarm reminders
Chat	ChatRepository, Firestore real-time listeners	ChatFragment, MessageAdapter, channel UI	—

Notifications	FCM token management, server-side triggers	Notification settings UI	FCMService, deep links
AI Chatbot	—	Chatbot UI (BottomSheet / dedicated screen)	Gemini API, RAG pipeline, knowledge Firestore
Campus Map	—	—	Google Maps SDK, building pins, navigation, AR overlay
Reviews	ReviewRepository, rating data model	Star rating UI, comment list, review form	Rating aggregation statistics
Wow Features	XP transactions, Similarity Check embeddings	PWr Exchange UI, Live Feedback charts	AI Tutor, Study Match, GPS Alarms, Dashboard
DevOps	GitHub Actions CI, Firebase env config	Screenshots, README content	Demo video, PDF report

4. DATABASE SCHEMA – FIRESTORE

The table below shows the complete Firestore collection schema. Document names use Firebase auto-ID or defined keys. All dates stored as Timestamp, files as Storage URLs referenced in documents.

Collection	Key Fields	Description
users/{uid}	role, name, email, faculty, avatar, fcmToken	User profile; role: admin lecturer student
courses/{courseId}	title, code, lecturerId, credits, tags, capacity	Course with description and metadata
courses/{id}/materials	fileName, url, uploadedAt, type (pdf video link)	Didactic materials uploaded by lecturer
courses/{id}/assignments	title, description, dueDate, maxScore, allowedTypes	Tasks to be completed by students
courses/{id}/assignments/{aid}/submissions	studentId, fileUrl, submittedAt, grade, feedback	Student submissions + grades
courses/{id}/announcements	message, createdAt, authorId, priority	Course channel announcements
enrollments/{docId}	studentId, courseId, enrolledAt, status, grade	Student course enrollments
schedules/{docId}	courseId, dayOfWeek, startTime, endTime, room, building	Timetable entries with room location
chats/{chatId}	participants[], courseId?, type (direct channel), lastMessage	Conversation metadata
chats/{id}/messages	senderId, text, timestamp, fileUrl?, readBy[]	Real-time messages
reviews/{docId}	targetType, targetId, rating, comment, authorId, createdAt	Ratings for courses, lecturers, buildings
buildings/{buildingId}	name, code, lat, lng, floors, description, photoUrl	PWr campus buildings for map pins
knowledge/{docId}	category, title, content, source, updatedAt, tags	AI chatbot knowledge base (FAQs, rules, maps)
userXP/{uid}	totalXp, weeklyXp, badges[], courseXp{}	Gamification: XP points and earned badges
studyGroups/{groupId}	participants[], courseId, status, matchScore, createdAt	Study Match groups with mutual acceptance
exchangeListings/{docId}	authorId, type, courseId, fileUrl, title, rating, price	PWr Exchange: notes, old exams, tutoring ads

globalAnnouncements	title, message, createdAt, authorId, targetRoles[]	Admin announcements to all roles
----------------------------	--	----------------------------------

4.1 Firestore Security Rules – Principles

- users/{uid}: read own document; write only by owner or admin
- courses: read for all authenticated users; write by lecturer (own courses) or admin
- submissions: read by student (own) + course lecturer; write by student (own only)
- chats/messages: access only for conversation participants (participants[])
- knowledge: read for all authenticated; write by admin only
- buildings: read public (authenticated); write by admin only
- userXP: read own; write via server-side transaction or trusted client logic

5. MILESTONE SCHEDULE – 3-DEVELOPER BREAKDOWN

Milestone	Due Date	Backend/Auth	UI/Courses	Chat/Maps/AI
M1	11 Mar 26	Technical proposal, Firestore schemas, Firebase project setup	UI wireframes, screen designs per role	PWr campus map research, chatbot API prototype
M2	25 Mar 26	Firebase Auth, role-based routing, user profile in Firestore	Login/register screens, role-specific navigation	Gemini/OpenAI API integration – chatbot PoC
M3	15 Apr 26	Course data models, enrollment logic, schedule repository	Course browse UI, enrollment flow, materials list	Campus map – building pins, search, InfoWindows
M4	13 May 26	Course management (admin/lecturer), timetable storage	Create/edit course UI, weekly timetable view	Review system – star ratings, comments, aggregation
M5	03 Jun 26	Assignments, Firebase Storage integration, access control	Assignment UI, file upload, student submission list	ICS export, FCM notifications, XP/badge system
M6	17 Jun 26	Firestore security rules, integration testing	Real-time chat (direct & channel), chat UI	AI Tutor, Study Match, GPS alarms, Similarity Check
Final	24 Jun 26	KDoc documentation, README, security review	Final UI polish, E2E tests, screenshots	Live demo, PDF report, GitHub release tag

5.1 Team Workflow

- Repository: GitHub with branch strategy – main, develop, feature/*, fix/*
- Code review: every PR requires review from at least one other team member
- Communication: daily standups (15 min), GitHub Projects for task tracking
- Environment: Google Firebase Console, Android Studio Hedgehog+, minSDK 26
- Versioning: semantic versioning (v1.0.0-milestone1, etc.)

6. COMPLETE FEATURE LIST

The table below lists all planned features with priority levels: **REQUIRED** (MAD project requirements), **PWr BONUS** (PWr-specific extensions), **OPTIONAL** (nice-to-have), and **WOW FEATURE** (advanced additions described in detail in Section 12).

Feature	Description	Priority
Firestore Auth + role-based routing	Login/register, redirect to correct screen per role	REQUIRED
Course management	Create, edit, delete courses by lecturer and admin	REQUIRED
Course enrollment	Student browses and enrolls in courses	REQUIRED
Didactic materials	Upload/download PDF, video, links per course	REQUIRED
Assignments + submission	Lecturer creates assignments, student uploads solution	REQUIRED
Real-time chat	Direct (1:1) or channel-based (per course) messaging	REQUIRED
FCM notifications	Push for deadlines, announcements, new messages	REQUIRED
PWr AI Chatbot	Gemini API + RAG with PWr knowledge base in Firestore	PWr BONUS
Interactive campus map	Google Maps with building pins A1-D3, room search, navigation	PWr BONUS
Review & rating system	Star ratings for courses, lecturers, buildings with comments	PWr BONUS
Advanced timetable	Weekly view, ICS export, 15-min reminders before class	PWr BONUS
Course materials management	Lecturer uploads PDFs, videos, links, notes per course	PWr BONUS
ICS calendar export	Export timetable to Google Calendar / Apple Calendar	OPTIONAL
QR room attendance	Scan QR on entry, attendance logged in Firestore real-time	OPTIONAL
AR building overlay	ARCore camera overlay with building labels on campus	OPTIONAL
Dark Mode + Material You	Dynamic colors (Android 12+), static dark theme fallback	OPTIONAL
Offline cache	Browse courses and materials without internet connection	OPTIONAL
Global search	Search courses, lecturers, buildings, deadlines by keyword	OPTIONAL
Home screen widget	Widget showing next class and unread message count	OPTIONAL
Deep links + Share	Share course/announcement links that open in-app	OPTIONAL
AI Tutor – personal mentor	Analyzes results daily, sends plan: 'Today: 2h calculus – exam in 4 days, here are 3 exercises'	FEATURE

Contextual GPS alarms	Calculates walking time to room, pushes: 'Leave now – 12 min to D3/215'	FEATURE
Study Match	Algorithm pairs students for study groups: same courses, similar grades, location, schedule	FEATURE
PWr Exchange	Marketplace for notes, old exams, tutoring ads from senior students	FEATURE
Live Feedback Dashboard	30-sec anonymous survey after each lecture; lecturer sees real-time comprehension charts	FEATURE
Similarity Check (plagiarism AI)	AI compares submissions via embeddings, flags suspicious pairs to lecturer	FEATURE
XP + Academic Badges	Points for activity: early submissions, attendance, grades. Rankings & badges like 'Calculus Guru'	FEATURE

7. PWr AI CHATBOT – DETAILS

7.1 Concept & Architecture

The PWr Chatbot is an intelligent assistant powered by Google Gemini 1.5 Flash (free tier API) using a RAG (Retrieval-Augmented Generation) technique. This means the LLM model is enhanced by a PWr-specific knowledge base stored in Firestore.

7.2 Knowledge Base – Firestore 'knowledge' Collection

- University regulations and grading rules
- Building and room locations (A1, B2, C3, D3, etc.) with room maps
- Study plans per faculty and semester
- Dean's office and secretariat contacts
- FAQ: course enrollment, exam dates, thesis procedures
- PWr news (updateable by admin in real time)

7.3 RAG Query Flow

1. Student types a question in the ChatbotFragment
2. App retrieves relevant documents from knowledge collection (Firestore query by tags)
3. Context + student question sent to Gemini API with a PWr-specific system prompt
4. Gemini generates a response incorporating PWr context
5. Answer displayed in UI with option for follow-up questions

API Limits

Google Gemini 1.5 Flash: 15 requests/min, 1M tokens/min on the free tier – sufficient for an academic project.

8. INTERACTIVE CAMPUS MAP

The interactive PWr campus map is implemented using Google Maps SDK for Android. Buildings are stored in the Firestore 'buildings' collection with GPS coordinates, photos, and room lists.

8.1 Map Features

- Pins for all campus buildings (A1, B1-B7, C1-C5, D1-D3, Library, Student Dorms)
- Tap a pin -> InfoWindow with building name, code, photo, room list
- Search: type room number or building -> map auto-centers on result
- Navigation: 'Take me there' button -> opens Google Maps with walking route
- Filter: show only buildings with the student's classes on a given day
- Optional: AR overlay (ARCore) with building labels through the camera

9. REVIEW & RATING SYSTEM

Students and lecturers can submit reviews for courses, lecturers, buildings, and rooms. The system aggregates ratings and displays averages on the relevant screens.

9.1 Review Types

- Courses: 1-5 stars + text comment, tags (interesting, difficult, recommended, etc.)
- Lecturers: rating for availability, communication quality, material value
- Buildings & rooms: accessibility (elevators, ramps), air conditioning, cleanliness
- Global announcements: emoji reactions (like, helpful, etc.)

9.2 Moderation

- Admin can remove reviews that violate community guidelines
- Each review is linked to userId – editable and deletable by author
- Firestore Security Rules block reviews from students not enrolled in a course

10. OPTIONAL BONUS FEATURES

10.1 QR Room Attendance

Each room has a unique QR code generated by admin. Student scans on entry -> app logs attendance in Firestore. Lecturer sees the real-time list of present students during class.

10.2 Android Home Screen Widget

The app can provide a home screen widget displaying the student's next upcoming class and unread message count. Implemented via AppWidgetProvider.

10.3 Inline Quizzes & Polls

Lecturer creates a short quiz (up to 5 questions) in the course channel. Students answer directly in the app. Results stored in Firestore and displayed to the lecturer as statistics.

10.4 Lecturer Analytics Dashboard

Dashboard for lecturers with charts: enrollment numbers per course (MPAndroidChart), average grades, student activity statistics (Firestore aggregations).

10.5 Room Overcrowding Crowdsourcing

If a room is overcrowded, students can report the issue in the app. Admin or lecturer receives alerts and can reassign the room. Feature based on a roomReports collection in Firestore.

10.6 Dark Mode & Material You

Full dark mode support with dynamic Material You colors (Android 12+). Static dark theme fallback with PWr colors for older Android versions.

12. FEATURES – DETAILED SPECIFICATIONS

The following 7 features go beyond the MAD project requirements and represent advanced extensions that significantly increase the value of the application for the entire PWr academic community.

12.1 AI Tutor – Personalized Study Mentor

Every morning, AI analyzes the student's situation and generates a personalized study plan. It knows what you are behind on, how much time you have before exams, and how you learn best.

How It Works

6. AI fetches the student's grade history from all courses in Firestore
7. Analyzes upcoming assignment and exam deadlines (assignments collection)
8. Gemini API generates a daily plan: 'Today: 2h calculus – quiz in 4 days. Here are 3 exercises tailored to your level.'
9. WorkManager triggers computation daily at 7:30 AM
10. FCM sends a push at 8:00 AM with a plan summary + deep link to full view

AI Input Data

- Grades and assignment results from Firestore (submissions collection with grade field)
- Assignment and exam deadlines (assignments collection, dueDate field)
- Available course materials (materials collection)
- Student activity timestamps (learning preference analysis)

Technology

- Google Gemini 1.5 Flash API – study recommendation generation
- WorkManager – daily background trigger without draining battery
- Firestore – input data and AI response cache (TTL: 24h)
- FCM – morning push notification with daily plan

Bonus

AI can automatically generate flashcards from PDFs uploaded by the lecturer using the Gemini document understanding API.

12.2 Contextual GPS Alarms – 'Leave Now'

Instead of a static alarm at time X, the app knows where you are and calculates exactly when you need to leave to make it to class on time. No more being late.

Flow

11. Student has class at 10:00 AM in room D3/215
12. App gets GPS location (Geofencing API – no continuous tracking)
13. Google Maps Directions API calculates pedestrian walking time to building D3
14. WorkManager schedules push: `class_time` minus `walking_time` minus 2-min buffer
15. Student gets notification: 'Leave now – Mathematics at 10:00 in D3/215, you have 12 minutes'

Smart Context Awareness

- Rain (WeatherAPI) – adds 2 minutes and suggests alternative route
- Student already in building (geofence triggered) – alarm auto-cancelled
- First time in this room – extra buffer added to find the room

Technology

- Android Geofencing API – location monitoring without battery drain
- Google Maps Directions API (Pedestrian mode) – walking time calculation
- WorkManager + AlarmManager – precise notification scheduling
- FCM + local notifications – displayed even when app is closed

12.3 Study Match – Study Group Algorithm

Study Match pairs students for study groups like a recommendation algorithm — based on shared courses, similar grade levels, preferred hours, and campus proximity.

Matching Algorithm

- Step 1: Filter – only students enrolled in the same courses
- Step 2: Scoring – grades (30%), time preferences (25%), location (25%), study goals (20%)
- Step 3: Suggestion – 'Kasia and Marek also have a math exam next week – want to study together?'
- Step 4: Mutual acceptance from both sides – group chat automatically created

Privacy & Security

- Matching is anonymous until mutual acceptance
- Student can disable Study Match in profile settings
- Match history and rejections stored locally (Room DB)

Technology

- Kotlin – scoring algorithm implementation on Firestore data
- Firestore – studyGroups collection with participants and status
- Existing chat infrastructure – group conversation after matching
- FCM – notification about a new match proposal

Post-exam bonus

App shows: 'Your Study Match group scored an average of 1.2 points higher than the rest of the year!' – motivation for next semester.

12.4 PWr Exchange – Knowledge & Tutoring Marketplace

A platform for exchanging academic materials between students. Senior students share their notes, summaries, and old exams, and offer tutoring for courses they have already passed.

Resource Types

- Notes & summaries: PDF/DOCX tagged by course and academic year
- Old exams/quizzes: require lecturer approval before publication
- Tutoring ads: listing with price (or free), available times, location (room / online)
- Diagrams & mind maps: graphic materials and visual study aids

Reputation System

- Each resource rated by downloaders (1-5 stars + comment)

- Each tutor rated after sessions by the student
- 'Trusted Mentor' badge after 10 positive tutoring ratings
- Top resources promoted in course search results

Technology

- Firebase Storage – resource file storage
- Firestore – exchangeListings collection with metadata and ratings
- Firebase Storage Security Rules – only author can delete their resource
- Full-text search via Firestore tag queries

12.5 Live Feedback Dashboard – Lecture Comprehension

After each lecture, the app automatically sends students a 30-second anonymous survey. The lecturer sees real-time charts: which topics were unclear, what percentage of the class followed along, and who submitted current assignments.

Survey Flow

16. Lecturer taps 'End Lecture' in the app
17. FCM push sent to all enrolled students: '30-second survey about today's lecture'
18. Student answers 3 emoji-scale questions
19. Results collected anonymously in Firestore within minutes
20. Lecturer sees real-time dashboard with charts

Dashboard Contents

- Pie chart: percentage of students who followed the lecture
- Topic heatmap: which subjects were unclear
- Indicator: how many students submitted current assignment vs. how many did not
- Trend line: comprehension across the last 5 lectures
- Word cloud from anonymous text comments (optional survey field)

Technology

- Firestore real-time listeners – instant results display
- MPAndroidChart – bar chart, pie chart, line chart
- FCM – triggered surveys on lecture end
- Firestore aggregations – average calculation per question

AI Summary

Gemini API automatically summarizes survey results and recommends: 'Return to the topic of integrals – 68% of students rated it as unclear.'

12.6 Similarity Check – AI Plagiarism Detection

When students upload text submissions, AI compares them against each other within the course and calculates a similarity score. The lecturer sees flagged suspicious pairs before reading anything.

Detection Flow

21. Student uploads a submission (PDF/DOCX) – text extracted automatically
22. Text converted to a vector embedding via Gemini Embeddings API
23. Embedding compared with other course submissions using cosine similarity

- 24. If similarity > 75% – 'Requires Verification' flag raised for lecturer
- 25. Lecturer view: 'Kowalski submission is 87% similar to Nowak – click to compare'

Important UI Disclaimers

- The system does NOT accuse – it only signals for human verification
- Results visible to course lecturer only, never to students
- Flagging threshold configurable by lecturer (default: 75%)
- Optional: AI-generated text detection score (AI authorship indicator)

Technology

- Gemini Embeddings API – text vector generation
- Firestore – embedding storage per submission
- Kotlin – cosine similarity implementation on vectors
- Firebase Storage – original files available for side-by-side comparison

12.7 XP + Academic Badges – Gamification System

An XP points and badge system motivates students through game-like mechanics familiar from educational apps. Every desired academic activity is rewarded.

XP Point Table

- Submitting an assignment early: +50 XP (bonus increases the earlier you submit)
- 100% attendance in a week: +100 XP
- Grade 5.0 on an assignment: +200 XP
- First answer in a course chat: +20 XP
- Uploading a resource to PWr Exchange: +75 XP
- Helping another student (confirmed by recipient): +50 XP

Example Badges

- 'Early Bird' – submitted assignment >24h before deadline 5 times
- 'Calculus Guru' – grade 5.0 on 3 mathematics/calculus assignments
- 'Mentor' – 10 positive tutoring ratings on PWr Exchange
- 'Always Present' – 100% attendance for an entire month
- 'Expert' – top 10% in course leaderboard

Leaderboards & Display

- Per-course leaderboard – visible to all enrolled students
- Faculty ranking – anonymous (initials only)
- Student profile shows earned badges and total XP
- Lecturer sees engagement ranking for their course

Technology

- Firestore transactions – atomic XP updates (prevent race conditions)
- userXP collection with fields: totalXp, badges[], weeklyXp, courseXp{}
- Client-side triggers after actions – XP granted after each event
- RecyclerView + Glide – leaderboard and badge display in profile

University Integration

Top students of the semester can receive physical rewards: discounts at the campus cafe or priority enrollment in popular courses.

11. FINAL SUBMISSION REQUIREMENTS (24 JUNE 2026)

11.1 GitHub Repository

- Clear README with project description, screenshots, run instructions
- google-services.json NOT committed – README explains how to configure Firebase
- Git tags per milestone (v0.1-milestone1 ... v1.0-final)
- GitHub Actions: automatic lint + build on each PR

11.2 PDF Report

- Architecture: MVVM diagram, layer descriptions
- Firestore schema with collection and relationship descriptions
- Firebase security rules overview
- List of implemented features (required + optional + wow features)
- AI Chatbot description and knowledge base overview
- Testing instructions: test accounts per role, demonstration flow

11.3 Live Demo

- Prepare 3 test accounts: admin@pwr.edu.pl, lecturer@pwr.edu.pl, student@pwr.edu.pl
- Demo scenario: student enrolls in course -> lecturer publishes assignment -> student submits -> push notification -> chat -> chatbot answers room question -> campus map -> leave review -> AI Tutor shows daily plan